

A. **Film Category Revenue Analysis Report**

This report identifies which film categories generate the highest revenue from customer rentals. It will inform inventory purchasing decisions and guide promotional strategies by identifying the most profitable genres. The analysis uses rental and payment data to calculate total revenue per category and present detailed records of individual transactions.

1. **Detailed table fields:**

- rental_id
- rental_date
- customer_id
- first_name / last_name
- title (from film)
- category_name (from category)
- amount (from payment)
- staff_id (optional for identifying the employee)

Summary table fields:

- category_name
- total_revenue (SUM of payment amounts)
- number_of_rentals (COUNT of rentals per category)
- average_revenue_per_rental (calculated field)

2. **Types of data fields used**

- Integer – rental_id, customer_id, staff_id
- Date/Time – rental_date
- String/Text – customer name, film title, category name
- Decimal – payment amount
- Derived/Numeric – total revenue, average revenue per rental

3. **Tables used**

- **payment** – provides rental revenue and amount
- **rental** – links payments to inventory and film
- **inventory** – connects rental to film
- **film** – gives you title and film_id
- **film_category** – maps film to category_id
- **category** – gives category name

4. Custom transformation with a user-defined function

Field: Customer full name

Transformation: Use a user-defined function (UDF) to concatenate `first_name` and `last_name` into a full name field.

Why: While the database stores names in separate fields, combining them improves readability and professionalism in business reports. A UDF can standardize this across different reports and future datasets.

5. Business use of detailed vs summary tables

- **Detailed table use:**

Stakeholders (e.g., store managers) can audit specific transactions, identify top spending customers, and monitor day to day performance of individual films or staff

- **Summary table use:**

Executives and analysts can quickly compare genre performance, spot trends, and make high level decisions about future film purchases or marketing pushes based on profitability

6. Refresh frequency

Suggested refresh rate: Monthly

Why:

Rental trends can shift with seasons, new releases, or customer behavior. A monthly refresh keeps insights up to date without being too burdensome, offering a solid balance between accuracy and performance.

B. CREATE OR REPLACE FUNCTION `get_full_name`(`first_name` TEXT, `last_name` TEXT)

RETURNS TEXT AS \$\$

BEGIN

 RETURN INITCAP(`first_name` || ' ' || `last_name`);

END;

\$\$ LANGUAGE plpgsql;

```
C. CREATE TABLE detailed_category_revenue (  
    rental_id INTEGER,  
    rental_date TIMESTAMP,  
    customer_id INTEGER,  
    full_name TEXT,  
    title TEXT,  
    category_name TEXT,  
    amount NUMERIC(5,2),  
    staff_id INTEGER  
);
```

```
CREATE TABLE summary_category_revenue (  
    category_name TEXT,  
    total_revenue NUMERIC(10,2),  
    number_of_rentals INTEGER,  
    average_revenue_per_rental NUMERIC(10,2)  
);
```

```
D. SELECT  
    r.rental_id,  
    r.rental_date,  
    c.customer_id,  
    get_full_name(c.first_name, c.last_name) AS full_name,  
    f.title,  
    cat.name AS category_name,  
    p.amount,  
    r.staff_id  
FROM rental r  
JOIN customer c ON r.customer_id = c.customer_id  
JOIN inventory i ON r.inventory_id = i.inventory_id  
JOIN film f ON i.film_id = f.film_id  
JOIN film_category fc ON f.film_id = fc.film_id  
JOIN category cat ON fc.category_id = cat.category_id  
JOIN payment p ON r.rental_id = p.rental_id;
```

E.

-- Step 1: Create the trigger function

```
CREATE OR REPLACE FUNCTION update_summary_category_revenue()
```

```
RETURNS TRIGGER AS $$
```

```
BEGIN
```

```
-- If the category already exists, update it
```

```
IF EXISTS (
```

```
    SELECT 1 FROM summary_category_revenue
```

```
    WHERE category_name = NEW.category_name
```

```
) THEN
```

```
    UPDATE summary_category_revenue
```

```
    SET
```

```
        total_revenue = total_revenue + NEW.amount,
```

```
        number_of_rentals = number_of_rentals + 1,
```

```
        average_revenue_per_rental =
```

```
            (total_revenue + NEW.amount) / (number_of_rentals + 1)
```

```
    WHERE category_name = NEW.category_name;
```

```
-- If it's a new category, insert it
```

```
ELSE
```

```
    INSERT INTO summary_category_revenue (
```

```
        category_name,
```

```
        total_revenue,
```

```
        number_of_rentals,
```

```
        average_revenue_per_rental
```

```
    )
```

```
    VALUES (
```

```
        NEW.category_name,
```

```
        NEW.amount,
```

```
        1,
```

```
        NEW.amount
```

```
    );
```

```
END IF;
```

```
RETURN NEW;
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```

-- Step 2: Create the trigger on the detailed table

```
CREATE TRIGGER trg_update_summary_category_revenue
```

```
AFTER INSERT ON detailed_category_revenue
```

```
FOR EACH ROW
```

```
EXECUTE FUNCTION update_summary_category_revenue();
```

```

F. CREATE OR REPLACE PROCEDURE refresh_category_revenue()
LANGUAGE plpgsql
AS $$
BEGIN
    -- Clear existing data
    TRUNCATE TABLE detailed_category_revenue;
    TRUNCATE TABLE summary_category_revenue;

    -- Repopulate detailed table (trigger will handle summary)
    INSERT INTO detailed_category_revenue (
        rental_id,
        rental_date,
        customer_id,
        full_name,
        title,
        category_name,
        amount,
        staff_id
    )
    SELECT
        r.rental_id,
        r.rental_date,
        c.customer_id,
        get_full_name(c.first_name, c.last_name) AS full_name,
        f.title,
        cat.name AS category_name,
        p.amount,
        r.staff_id
    FROM rental r
    JOIN customer c ON r.customer_id = c.customer_id
    JOIN inventory i ON r.inventory_id = i.inventory_id
    JOIN film f ON i.film_id = f.film_id
    JOIN film_category fc ON f.film_id = fc.film_id
    JOIN category cat ON fc.category_id = cat.category_id
    JOIN payment p ON r.rental_id = p.rental_id;

END;
$$;

```

F1. Relevant job scheduling tool:

pg_cron – A PostgreSQL extension for running SQL jobs on a schedule

H. No outside sources utilized for code